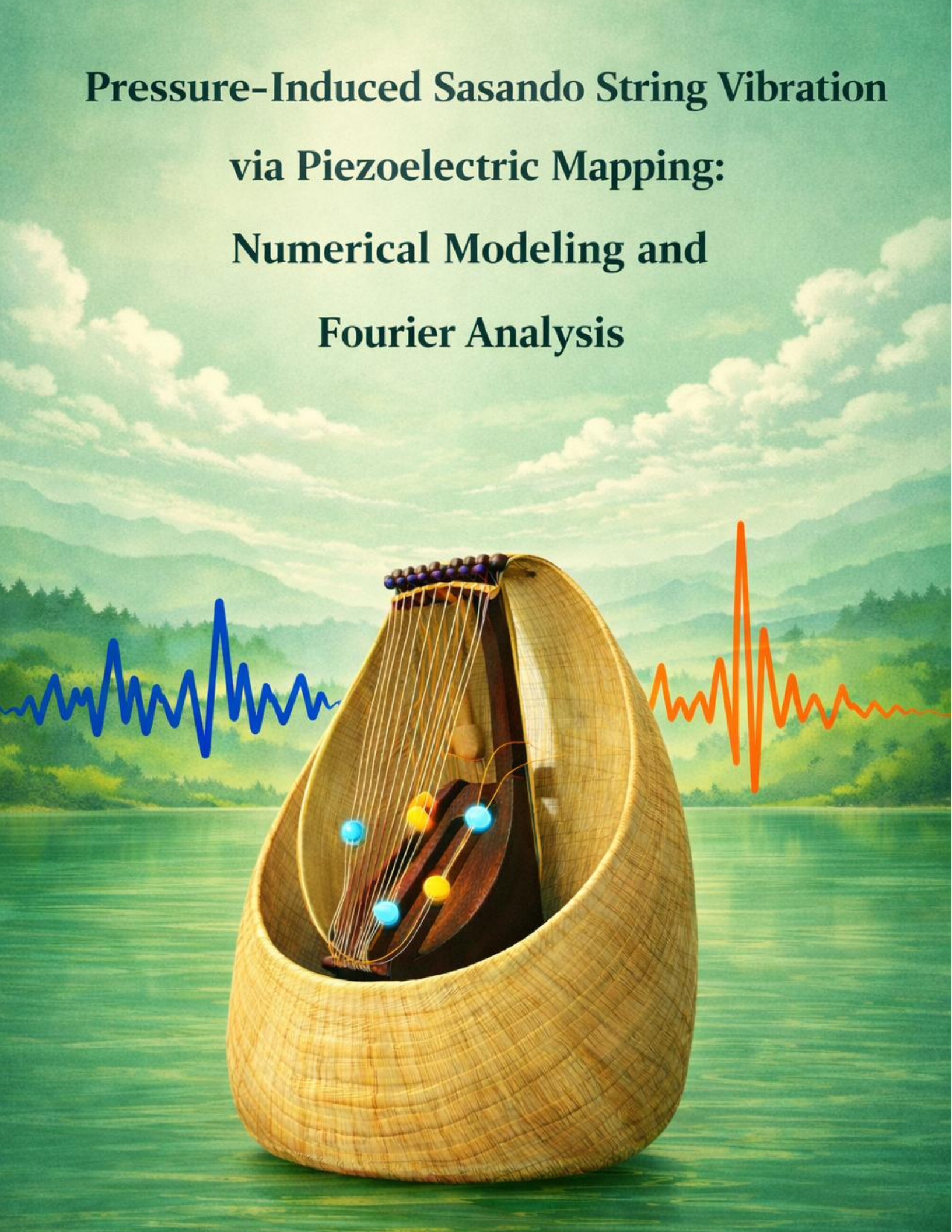
Pressure-Induced Sasando String Vibration via Piezoelectric Mapping: Numerical Modeling and Fourier Analysis



BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 212 (July 2025)

Numerical Technique Laboratory

Final Project Report

Pressure-Induced Sasando String Vibration via Piezoelectric Mapping: Numerical Modeling and Fourier Analysis

Submitted to:

Md. Ehsanul Karim
Lecturer, Department of EEE
Md. Mostaqul Islam
Part Time Lecturer, Department of EEE

Submitted by:

Project Group: 03
Section: C-2

Member Name	**Student ID**
Anirudha Roy Archo	2306175
Monem Tazuar Alve	2306176
Kushal Roy Sondo	2306177
Swapneel Paul	2306178

Table of Contents

1. Abstract 4
2. Introduction 5
3. Problem Formulation 6
   - 3.1 Identification of Scope 6
   - 3.2 Formulation of the Engineering Problem 6
   - 3.3 Analysis Overview 7
4. Mathematical Model of String Vibration 8
5. Energy-Based Derivation 8
6. Design Method 11
   - 6.1 Design Philosophy 11
   - 6.2 System Decomposition 11
   - 6.3 Pressure-to-Electrical Mapping 11
   - 6.4 Voltage-to-Length Mapping 12
   - 6.5 Frequency Generation Based on String Model 12
   - 6.6 Stability and RC Constraint Design 13
   - 6.7 Frequency-Based Filtering Segment 13
   - 6.8 Fourier Synthesis and Signal Construction 14
   - 6.9 Audio Signal Generation and Visualization 16
   - 6.10 Frequency Spectrum Analysis 17
   - 6.11 Pressure-to-Note Mapping 19
   - 6.12 MATLAB GUI-Based Implementation 19
7. Design Analysis and Evaluation 20
   - 7.1 Novelty 20
   - 7.2 Investigation 21
     - 7.2.1 Design of Experiment 21
     - 7.2.2 Data Collection 21
     - 7.2.3 Result and Analysis 26
     - 7.2.4 Interpretation and Conclusion on Data 27
     - 7.2.5 Model Validation 28
8. Limitations of Tools 27
9. Reflection on Individual 28
10. Conclusion 28
11. Future Work 28

References 31

1.Abstract

This project investigates the vibration behavior of a Sasando string under pressure-based excitation using numerical modeling and frequency-domain analysis. The physical motion of the Sasando string is modeled based on established partial differential equations governing string vibration. Pressure inputs are mapped to electrical and mechanical analog parameters, and resulting frequencies are analyzed using Fourier synthesis. A MATLAB-based graphical user interface is developed to visualize pressure-to-frequency mapping, audio generation, Fourier epicycles, and spectral behavior in real time. The system demonstrates stable frequency generation within RC-imposed physical limits and provides an interactive platform for understanding the relationship between pressure excitation, string vibration, and harmonic content.

2.Introduction

The Sasando is a traditional stringed musical instrument originating from Rote Island in Indonesia, known for its distinctive tonal quality produced by vibrating strings. Sound generation in the Sasando results from oscillatory motion of stretched strings when mechanical excitation is applied through plucking or pressure. The resulting vibration depends on physical parameters such as string length, tension, elasticity, and damping. Analyzing this vibration behavior is important from an engineering perspective, as string motion represents a classical wave-based physical system governed by fundamental laws of mechanics. However, due to the complexity of real instruments and interacting physical effects, direct experimental analysis alone is insufficient to fully explain the vibration dynamics of Sasando strings.

Mathematical modeling offers a systematic approach to represent Sasando string vibration using differential equations derived from physical principles. The motion of a stretched string can be expressed through partial differential equations that account for elastic restoring forces, damping, and external excitation. Analytical solutions to such equations are often limited, making numerical methods essential for studying transient behavior and stability. Furthermore, the acoustic output of a vibrating string consists of multiple frequency components, which can be effectively analyzed using Fourier techniques. In this project, MATLAB is used to integrate numerical modeling, pressure-to-frequency mapping, Fourier-based frequency synthesis, and real-time visualization through a graphical user interface. This approach enables interactive analysis of string vibration, harmonic content, and sound generation, providing a clear connection between physical excitation, numerical computation, and acoustic response.

3. Problem Formulation

3.1 Identification of Scope

The scope of this project is to analyze and visualize the vibration behavior of a Sasando string subjected to pressure-based excitation using numerical and signal-processing techniques. The system considers pressure as the primary external input, which is transformed into mechanical deformation, frequency generation, and acoustic output. The study focuses on modeling the relationship between applied pressure and the resulting vibration frequency while respecting physical constraints imposed by material properties and electrical filtering elements. The scope also includes time-domain and frequency-domain analysis of the generated signal using Fourier techniques, along with real-time visualization and audio synthesis through a MATLAB-based graphical user interface.

The project does not attempt to replicate the full physical construction of the Sasando instrument. Instead, it abstracts the string behavior into a mathematically and numerically tractable model that captures the essential dynamics of string vibration, frequency formation, and harmonic content. Emphasis is placed on numerical modeling, visualization, and interpretation rather than experimental fabrication or hardware implementation.

3.2 Formulation of the Engineering Problem

When pressure is applied to a Sasando string, mechanical energy is introduced into the system, causing the string to deform and vibrate. This vibration can be modeled using mathematical equations derived from classical wave mechanics, where the string motion depends on elasticity, damping, and boundary conditions. However, solving the resulting governing equations analytically becomes difficult due to damping effects and practical constraints. Additionally, the vibration of the string generates a complex signal composed of multiple frequency components, making frequency-domain analysis necessary for understanding sound characteristics. The engineering problem addressed in this project is to develop a numerical and computational framework that can convert pressure input into physically meaningful vibration frequencies, analyze the resulting harmonic structure, and generate

corresponding audio output. This requires integrating numerical modeling of string behavior with Fourier-based frequency synthesis while ensuring system stability through appropriate parameter constraints. The challenge lies in accurately mapping pressure values to frequencies, maintaining stability under filtering constraints, and providing intuitive visualization of both time-domain vibration and frequency-domain behavior.

3.3 Analysis Overview

To address the formulated problem, the system processes pressure inputs through a sequence of numerical transformations that represent physical behavior. Pressure values are first mapped to electrical and mechanical analog quantities, which are then converted into frequency components based on string vibration principles. Fourier analysis is applied to synthesize and visualize the resulting frequency components, allowing observation of individual harmonics and their combined effect. Stability considerations are incorporated by limiting frequency generation using resistance–capacitance (RC) constraints, ensuring that the synthesized signals remain physically realizable.

The effectiveness of the proposed approach is evaluated through MATLAB simulations and graphical visualizations, including frequency spectra, Fourier epicycle animations, and real-time audio playback. This structured analysis framework enables systematic investigation of the relationship between pressure excitation, string vibration, and acoustic output.

4. Mathematical Model of String Vibration

The vibration of a Sasando string can be modeled by considering the string as a continuous elastic medium subjected to external excitation and damping. Let $u(x,t)$ denote the transverse displacement of the string at spatial position x and time t. Based on classical wave theory and energy-based derivation, the motion of the Sasando string is governed by a second-order hyperbolic partial differential equation.

The general mathematical model for Sasando string vibration is given by

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} + k_d \frac{\partial u}{\partial t} = 0$$

where
$u(x,t)$ is the transverse displacement of the string,
c is the wave propagation speed related to string elasticity,
l is the effective string length, and
k_d is the damping constant that accounts for energy loss due to internal friction and air resistance.

This equation represents a damped wave system, where the first term corresponds to inertial effects, the second term represents elastic restoring forces, and the third term models damping effects. The presence of the damping term causes the vibration amplitude to decrease with time, leading the system toward an equilibrium state.

5.Energy-Based Derivation

The governing equation is derived using Lagrangian mechanics by defining the total kinetic and potential energies of the Sasando string. When the string is excited, both kinetic and potential energies are generated along its length.

The kinetic energy of the string is expressed as

$$E_k = \frac{1}{2}\rho l \left(\frac{\partial u}{\partial t}\right)^2$$

where ρ is the mass density of the string.

The total potential energy consists of three components: spring potential energy, stress potential energy, and frictional energy. The spring potential energy is given by

$$E_{p,spring} = \frac{1}{2} k_p l^2 \left(\frac{1}{2}\left(\frac{\partial u}{\partial x}\right)^2 + 1\right)$$

The stress-related potential energy due to string elasticity is

$$E_{p,stress} = 2E\left(\frac{1}{2}\left(\frac{\partial u}{\partial x}\right)^2 + 1\right)$$

where E is the Young's modulus of the string material.Energy dissipation caused by damping and friction is modeled as

$$E_{p,friction} = -k_b\eta\left(\frac{\partial u}{\partial t}\right)u$$

Combining all potential energy components, the total potential energy of the system becomes

$$E_p = \left(\frac{1}{2}k_p l^2 + 2E\right)\left(\frac{1}{2}\left(\frac{\partial u}{\partial x}\right)^2 + 1\right) - k_b\eta\left(\frac{\partial u}{\partial t}\right)u$$

The total potential energy combines elastic deformation, material stress, and damping effects. These energy terms collectively describe how the string stores and dissipates energy during vibration

The Lagrangian of the system is then defined as

$$\mathcal{L} = E_c - E_p$$

Applying the Euler–Lagrange equation to the Lagrangian function results in the governing partial differential equation describing the damped vibration of the Sasando string.

Initial and Boundary Conditions

To obtain a unique solution, appropriate initial and boundary conditions are imposed. The Sasando string is assumed to be fixed at both ends, leading to the boundary conditions

$$u(0,t) = 0, \quad u(l,t) = 0$$

The initial displacement of the string is modeled as a triangular function representing pressure-based excitation:

u(x,0) = f(x)

where

$$f(x) = \begin{cases} \frac{2hx}{l}, & 0 \le x \le \frac{1}{2}l \\ 2h - \frac{2hx}{l}, & \frac{1}{2}l \le x \le l \end{cases}$$

The Sasando string is assumed to be fixed at both ends. Therefore, the displacement at both boundaries remains zero for all time.

$$\frac{\partial u}{\partial t}(x, 0) = 0$$

These conditions represent a realistic scenario in which the Sasando string is initially displaced due to applied pressure and then released, allowing free vibration to occur.

Relation to Frequency Generation

The solution of the string vibration model yields oscillatory motion whose dominant frequency corresponds to the fundamental frequency of the string. For a stretched string, the fundamental frequency is related to tension T, string length L, and mass density μ by

$$f = \frac{1}{2L}\sqrt{\frac{T}{\mu}}$$

In this project, pressure input is mapped to effective string tension, allowing numerical computation of vibration frequency. These frequencies are later used for Fourier synthesis, spectral analysis, and audio generation within the MATLAB environment

6.Design Method

6.1 Design Philosophy

The design method of this project is based on the systematic application of mathematical modeling, numerical computation, and signal processing principles to analyze Sasando string vibration under pressure excitation. The objective is to convert physical pressure input into meaningful vibration frequencies and acoustic output while ensuring numerical stability and physical realism. MATLAB is selected as the primary platform due to its strong capabilities in numerical computation, visualization, and graphical user interface (GUI) development.

6.2 System Decomposition

To achieve a structured and modular design, the overall system is decomposed into sequential processing stages. Each stage represents a transformation of physical or numerical variables, ensuring clarity and ease of implementation. The design follows a unidirectional signal flow, starting from pressure input and ending at audio and spectral visualization.

The processing flow of the system is given by:

When a piezoelectric material is mechanically strained, electric charges appear on the surfaces of the crystal. When the direction of the strain is reversed, the polarity of the electric charges is also reversed. This phenomenon is called the **direct piezoelectric effect**, and the materials that exhibit this behavior are known as **piezoelectric crystals**.

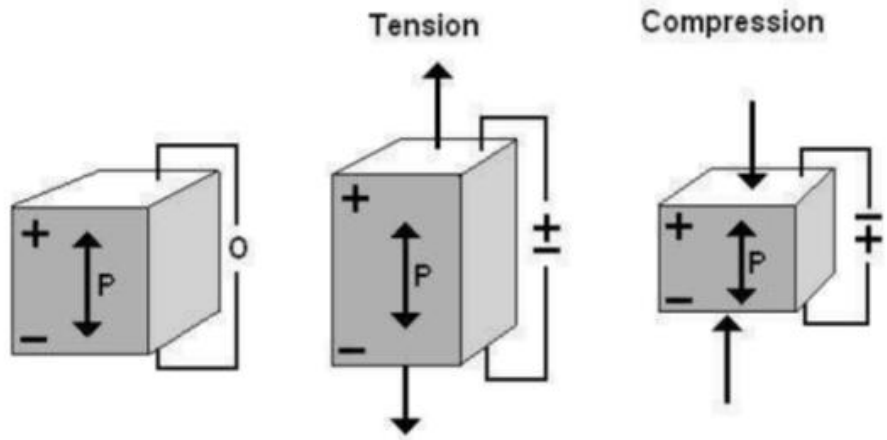


Figure 1. The direct piezoelectric effect.

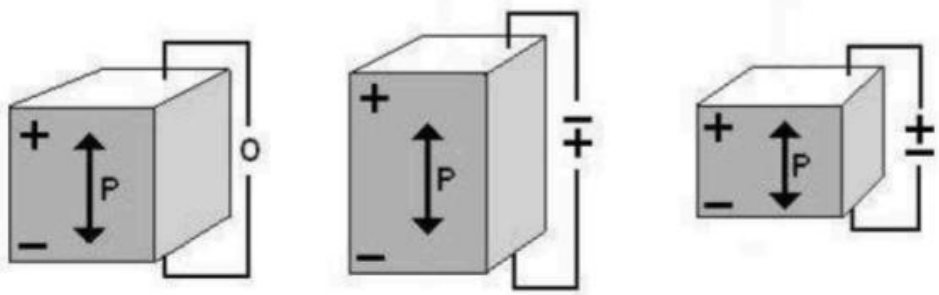


Figure 2. The converse piezoelectric effect.

6.3 Pressure-to-Electrical Mapping

The first stage of the design converts applied pressure into an equivalent electrical quantity. This abstraction allows pressure to be processed numerically while preserving proportionality to physical excitation. A piezoelectric-inspired relation is used to map pressure to voltage, where pressure input acts as the driving stimulus for the system. This mapping provides a controllable and scalable interface between physical input and numerical processing

$$V \propto P$$

CODE

```
function v = ptoV(p)
    d  = 400e-12;
    t  = 5e-4;
    eo = 8.854e-12;
    er = 2000;
    v = (d .* p .* t) ./ (er * eo);
end
```

6.4 Voltage-to-Length Mapping

The second stage of the design converts the electrical signal into a corresponding mechanical displacement. This abstraction allows voltage to be represented as a physical length while maintaining proportionality with the electrical excitation. An actuator-inspired relation is used to map voltage to length, where the voltage input serves as the driving signal for the system. This mapping establishes a controllable and scalable interface between electrical input and mechanical output.

$$L \propto V$$

CODE

```
function L = vtol(v)
    k = 1e4;
    L = v ./ k;
end
```

6.5 Frequency Generation Based on String Model

The mechanical displacement is used to determine the vibration frequency of the Sasando string based on classical string vibration theory. According to the governing principles of stretched strings, the vibration frequency depends on effective string tension, length, and mass density. In the proposed design, pressure-controlled displacement influences effective tension, allowing pressure values to be mapped directly to frequency components suitable for sound synthesis.

$$f = \frac{1}{2L}\sqrt{\frac{T}{\mu}}$$

CODE

```
function f = Ltof(L)
    Ls = 0.7;
    E  = 5e9;
    A  = pi*(0.8e-3)^2;
    mu = 7.3e-3;
    T = (E * A / Ls) .* L;
    f = (1./(2*Ls)) .* sqrt(T ./ mu);
end
```

After this an extra frequency that comes from the bamboo part of the instrument will add to the previous frequency of the system. So

F2=F+501.6, WHERE **501.6** Hz is the bamboo frequency

6.6 Stability and RC Constraint Design

To ensure numerical stability and prevent physically unrealistic frequency generation, resistance–capacitance (RC) constraints are incorporated into the design. The RC network imposes an upper cutoff frequency, limiting the maximum allowable vibration frequency generated by the system. This constraint acts as a stability filter, ensuring that only frequencies within a physically realizable range are passed to the synthesis stage.

$$f_c = \frac{1}{2\pi RC}$$

CODE

```
HPF = 1/(2*pi*R*C);
LPF = 98;
```

6.7 Frequency-Based Filtering Segment

In this stage, a filtering mechanism is applied to isolate the relevant frequency components of the signal. A high-pass filter (HPF) cut-off frequency is first defined using the relation:

A predefined low-pass threshold is also introduced to limit the upper bound of the frequency range. Using these two constraints, an index mask is generated to select only those frequency components that satisfy:

- Frequency greater than the high-pass cutoff
- Frequency greater than the low-pass threshold

This ensures that only significant signal components within the desired operating range are retained for further processing.

To maintain robustness, a fallback condition is implemented. If no frequency components satisfy the initial filtering criteria, an adaptive range is applied based on the signal's maximum and minimum frequency values. Specifically, frequencies within 90% to 110% of the observed range are selected. This guarantees that meaningful data is always preserved, even under varying or unexpected signal conditions.

CODE

```
HPF = 1/(2*pi*R*C);
LPF = 98;

IDX = (F2 < HPF) & (F2 > LPF);
if sum(IDX) == 0
    IDX = (F2 < max(F2)*1.1) & (F2 > min(F2)*0.9);
end
```

6.8 Fourier Synthesis and Signal Construction

Once valid frequency components are obtained, Fourier synthesis is used to construct the resulting vibration signal. Each frequency component is represented as a sinusoidal function with amplitude proportional to the corresponding pressure input. The superposition principle is applied to sum all sinusoidal components,

producing a composite signal that represents the overall vibration and acoustic output of the Sasando string.

$$x(t) = \sum_{k=1}^{N} A_k \, sin \, sin \, (2\pi f_k t)$$

```
function runFourier(P, R, C)
    [FILTERED_F2, FILTERED_P, freqs, amps] = processData(P, R, C);

    N_freqs = length(freqs);
    anim_time = 4;
    anim_samples = 200;
    anim_x = linspace(0, anim_time*2*pi, anim_samples);

    figure('Position', [50 50 1400 700], 'Color', [0.1 0.1 0.2]);

    color_map = hsv(N_freqs);
    components = zeros(N_freqs, length(anim_x));
    for k = 1:N_freqs
        components(k, :) = amps(k) * sin(freqs(k) * anim_x / max(freqs) * 2);
    end
    composite_signal = sum(components, 1);

    for frame = 1:10:length(anim_x)
        clf;
        subplot(2, 2, [1, 3]);
        hold on; grid on; axis equal;
        cumulative_x = 0; cumulative_y = 0;
        for k = 1:N_freqs
            phase = freqs(k) * anim_x(frame) / max(freqs) * 2;
            vec_x = amps(k) * cos(phase);
            vec_y = amps(k) * sin(phase);

            theta_circle = linspace(0, 2*pi, 50);
            circle_x = amps(k) * cos(theta_circle);
            circle_y = amps(k) * sin(theta_circle);
            plot(circle_x + cumulative_x, circle_y + cumulative_y, '--', 'Color', color_map(k,:));

            line([cumulative_x, cumulative_x + vec_x], ...
```

```
            [cumulative_y, cumulative_y + vec_y], ...
            'Color', color_map(k,:), 'LineWidth', 2);

        cumulative_x = cumulative_x + vec_x;
        cumulative_y = cumulative_y + vec_y;
    end

    plot(cumulative_x, cumulative_y, 'o', 'MarkerSize', 12, 'MarkerFaceColor', [1 0.5 0], 'Marker

    xlim([-1.2 1.2]); ylim([-1.2 1.2]);
    title('Fourier Epicycles','Color',[1 1 1]);

    subplot(2, 2, 2);
    hold on; grid on;
    for k = 1:N_freqs
        plot(anim_x(1:frame), components(k, 1:frame), 'LineWidth', 1.5);
    end
    title('Frequency Components','Color',[1 1 1]);

    subplot(2, 2, 4);
    plot(anim_x(1:frame), composite_signal(1:frame), 'LineWidth', 2.5);
    title('Composite Signal','Color',[1 1 1]);

    drawnow;
end
```

6.9 Audio Signal Generation and Visualization

In this stage, the filtered frequency and amplitude components are transformed into a time-domain audio signal and subsequently visualized. The function begins by retrieving the processed frequency components and their corresponding amplitudes from the pre-processing stage. A standard audio sampling frequency of 44.1 kHz is selected, and a time vector is generated over a fixed duration.

The filtered amplitudes are first normalized with respect to their total sum to ensure proportional contribution of each frequency component. A scaling factor is then applied to enhance the overall signal strength. Using these normalized amplitudes, the audio signal is synthesized as a superposition of sinusoidal waves, where each component frequency contributes according to its respective amplitude:

To prevent distortion or clipping during playback, the generated signal is normalized by its maximum absolute value. This ensures that the signal remains within an acceptable dynamic range.

For visualization, a sliding window approach is employed. A segment of the signal is plotted and continuously updated in real time, creating a dynamic waveform display synchronized with audio playback. The window advances in fixed step sizes, allowing smooth temporal progression of the signal. Simultaneously, an audio player object is used to reproduce the generated sound.

This integrated approach provides both auditory and visual representation of the processed data, enabling intuitive interpretation of the signal characteristics.

CODE

```
function runAudioVisual(P, R, C)
    [FILTERED_F2, FILTERED_P, freqs, amps] = processData(P, R, C);
    fs = 44100;
    T = 8;
    t = 0:1/fs:T;
    x = sum(FILTERED_P);
    amplitudes = FILTERED_P/x;
    amplitudes = amplitudes*1.5;
    audio_signal = zeros(size(t));
    for k = 1:length(FILTERED_F2)
        audio_signal = audio_signal + amplitudes(k) * sin(2*pi*FILTERED_F2(k)*t);
    end
    audio_signal = audio_signal / max(abs(audio_signal));
    figure('Position', [100 100 1200 500], 'Color', [0.1 0.1 0.2]);
    window_size = 2000;
    step_size = 100;
    num_frames = floor((length(audio_signal)-window_size)/step_size);
    h_plot = plot(t(1:window_size), audio_signal(1:window_size), 'LineWidth', 2);
    hold on; grid on;
    player = audioplayer(audio_signal, fs);
    play(player);
    for frame = 1:num_frames
        start_idx = (frame-1)*step_size + 1;
        end_idx = start_idx + window_size - 1;
        set(h_plot,'XData',t(start_idx:end_idx),'YData',audio_signal(start_idx:end_idx));
        xlim([t(start_idx) t(start_idx)+window_size/fs]);
        drawnow;
        if ~isplaying(player), break; end
    end
end
```

6.10 Frequency Spectrum Analysis

In this stage, the generated time-domain signal is analysed in the frequency domain using the Fast Fourier Transform (FFT). The signal is first normalized to avoid distortion, and then FFT is applied to obtain its spectral components. A single-sided amplitude spectrum is extracted by considering only positive frequencies and scaling appropriately.

The frequency axis is constructed based on the sampling frequency, ensuring accurate representation of real-world frequencies. Three types of visualizations are used for better understanding. A stem plot shows the dominant frequency components of the signal. A scatter plot represents the relationship between pressure and frequency, where amplitude is indicated through colour variation. Additionally, a line plot illustrates how amplitude changes with frequency.

This combined analysis provides a clear understanding of how the input signal is distributed across different frequencies and helps in identifying key spectral features effectively.

CODE

```
function runSpectrum(P, R, C)
    [FILTERED_F2, FILTERED_P, freqs, amps] = processData(P, R, C);
    fs = 44100;
    T = 8;
    t = 0:1/fs:T;
    x = sum(FILTERED_P);
    amplitudes = FILTERED_P/x;
    amplitudes = amplitudes*1.5;
    audio_signal = zeros(size(t));
    for k = 1:length(FILTERED_F2)
        audio_signal = audio_signal + amplitudes(k) * sin(2*pi*FILTERED_F2(k)*t);
    end
    audio_signal = audio_signal / max(abs(audio_signal));
    figure('Position',[100 100 1300 500],'Color',[0.1 0.1 0.2]);
    subplot(1,3,1);
    N = length(audio_signal);
    Y = fft(audio_signal);
    P2 = abs(Y/N);
    P1 = P2(1:N/2+1);
    P1(2:end-1)=2*P1(2:end-1);
    f = (0:(N/2))*fs/N;
    stem(f(1:2000),P1(1:2000),'Marker','none');
    title('Frequency Spectrum','Color',[1 1 1]);
    set(gca,'Color',[0.15 0.15 0.25],'XColor',[1 1 1],'YColor',[1 1 1]);
    subplot(1,3,2);
    scatter(FILTERED_P, freqs, 100, amps, 'filled');
    title('Pressure vs Frequency','Color',[1 1 1]);
    set(gca,'Color',[0.15 0.15 0.25],'XColor',[1 1 1],'YColor',[1 1 1]);
    colorbar;
    subplot(1,3,3);
    plot(FILTERED_F2, amps,'-o');
    title('Amplitude vs Frequency','Color',[1 1 1]);
    set(gca,'Color',[0.15 0.15 0.25],'XColor',[1 1 1],'YColor',[1 1 1]);
```

6.11 Pressure-to-Note Mapping for Sound Generation

This code defines a function that converts pressure input into musical notes using a SARGAM-based mapping system. It first retrieves frequencies and note names based on selected base scale and intonation values. The input pressure is then mapped to an index, which determines the corresponding note. The function plays this note using a fixed sampling rate along with user-defined duration and volume settings. It also updates a status label to display the pressure value and selected note, providing real-time feedback. Overall, the function creates an interactive system that connects physical pressure input with dynamic musical sound output.

CODE

```
    function playPressureNoteGUI(p)
        SARGAM = getSargam(baseSaField.Value, intonation.Value);

        idx = pressure_to_note(p);
        play_note(SARGAM.freqs(idx), 44100, durationField.Value, volumeField.Value);
        statusLabel.Text = sprintf('Pressure %d Pa → %s', p, SARGAM.names{idx});
    end
end
```

6.12 MATLAB GUI-Based Implementation

The complete design is implemented in MATLAB using a graphical user interface (GUI) to allow interactive control and visualization. Users can input pressure values and system parameters, observe generated frequencies, visualize Fourier components, and listen to synthesized audio output in real time. The GUI integrates numerical computation, graphical animation, and audio playback into a single interactive platform, enhancing both analysis and user understanding.

Pressure values are converted into voltage-like quantities, which are then transformed into effective mechanical displacement variables representing string deformation. These transformations allow the physical excitation to be represented in a numerically convenient form while preserving proportional relationships between pressure and vibration intensity.

CODE

```
function createGUI()
    fig = uifigure('Name','Sargam + Fourier Visualizer','Position',[200 100 1100 650]);
    fig.Color = [0.1 0.1 0.2];

    uilabel(fig,'Text','Sargam + Pressure Frequency Visualizer','Position',[240 590 650 40],...
        'FontSize',20,'FontWeight','bold','FontColor',[1 1 1]);
```

7. Design Analysis and Evaluation

7.1 Novelty

The proposed system provides an interactive numerical framework for analysing Sasando's string vibration under pressure excitation. Unlike traditional vibration models that focus solely on solving differential equations, this project integrates multiple engineering domains including mechanical vibration modelling, signal synthesis, and real-time visualization.

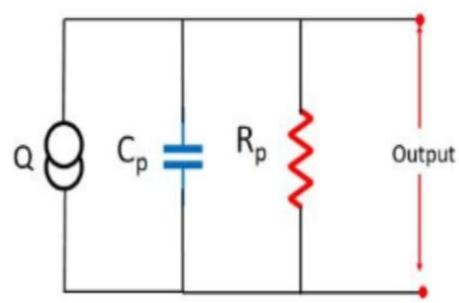
The key novelty of the system includes:

•Integration of pressure-driven excitation with string vibration modelling.
• Mapping between pressure input, mechanical deformation, and generated vibration frequency.
•Real-time Fourier synthesis of vibration signals.
•Visualization of harmonic components using Fourier epicycle animations.
•Interactive MATLAB GUI that combines signal generation, visualization, and audio playback.

By combining numerical modelling with signal-processing based sound synthesis, the system provides an intuitive platform for understanding the relationship between excitation forces, vibration physics, and acoustic output.

7.2 Investigations

7.2.1 Design of Experiment



Equivalent Circuit of Piezoelectric Transducer 1

Fig-03

7.2.2 Data Collection

For each simulation run, the system generates the following data:

- Input pressure values
- Generated vibration frequencies
- Time-domain composite signal
- Frequency spectrum obtained using FFT
- Fourier epicycle animation representing harmonic components

GUI Interface:

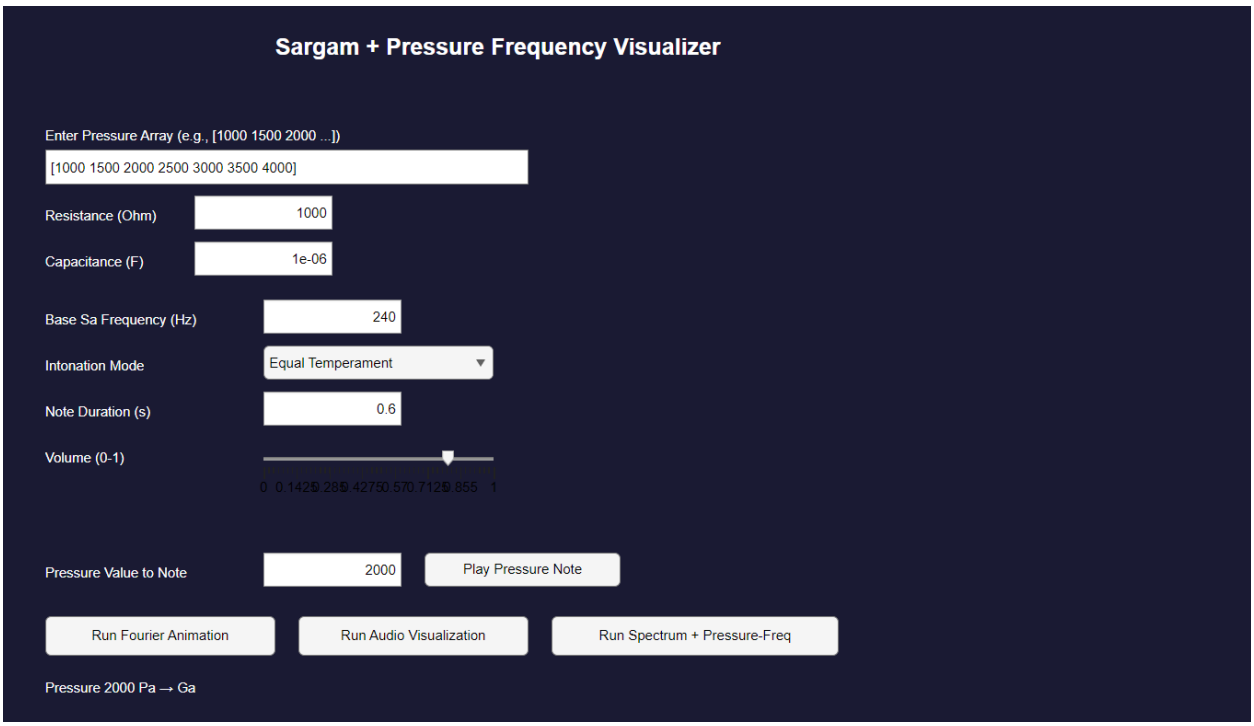


Fig. 04

Graphical outputs generated include:

1.Audio visualization:

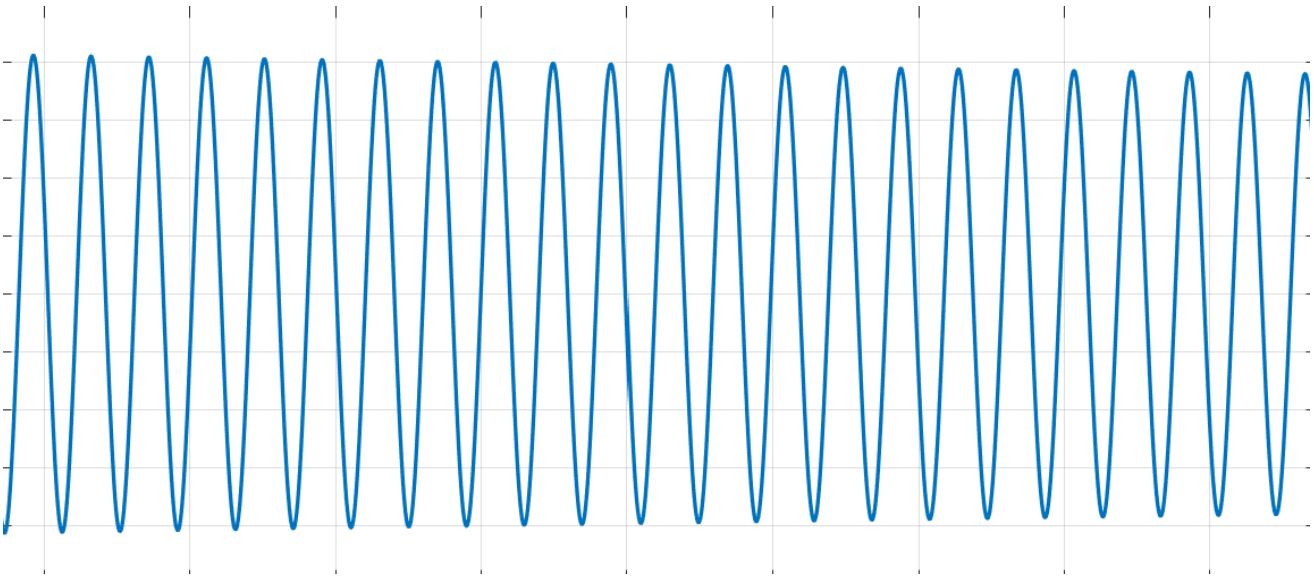


Fig. 05

From fig-01, a stable and periodic oscillation with no visible distortion. The smooth sinusoidal structure confirms that the Fourier synthesis and normalization processes are functioning correctly, producing a physically consistent audio signal.

Youtube link : https://www.youtube.com/watch?v=xSNBDG8yvukc

2. Frequency spectrum of the signal

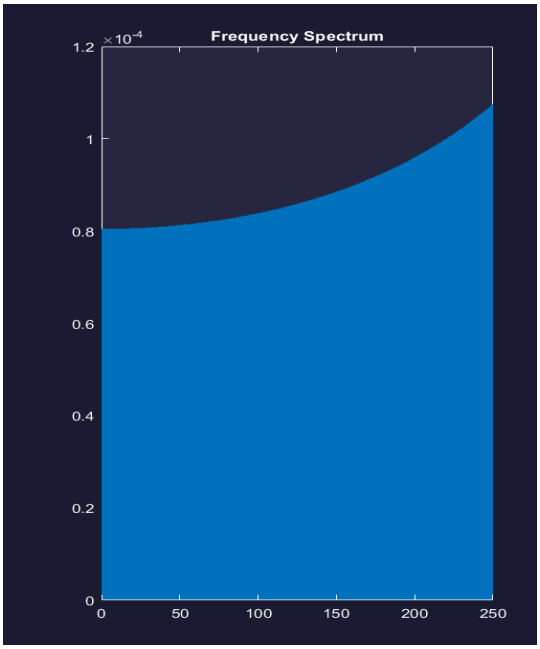


Fig. 06

The frequency spectrum in Fig. 2 shows a smooth amplitude distribution across frequencies, indicating a composite signal composed of multiple harmonic components.

3.Pressure vs Frequency plot

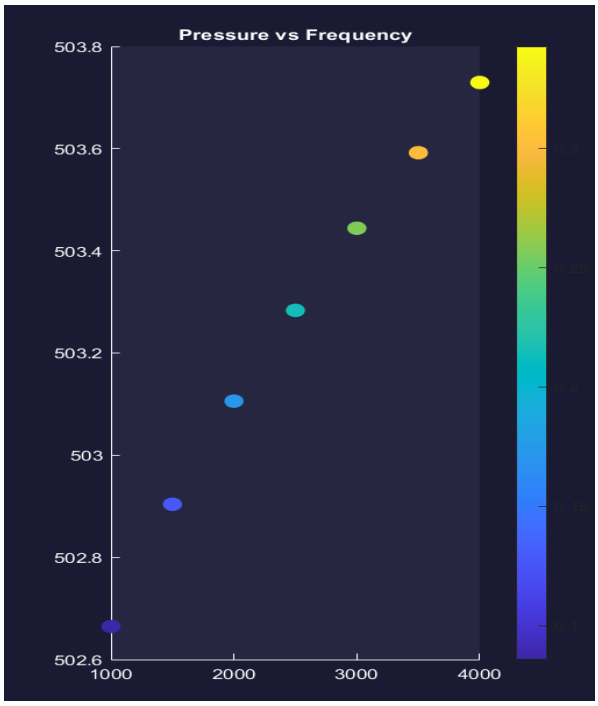


Fig-03

In fig-03, the P vs f plot demonstrates a linear trend from approximately 502.7 Hz to 503.7 Hz as pressure increases from 1000 Pa to 4000 Pa. This behavior is consistent with string vibration theory, where frequency increases with effective tension. The gradual slope indicates a nonlinear but stable relationship.

4• Amplitude vs Frequency plot

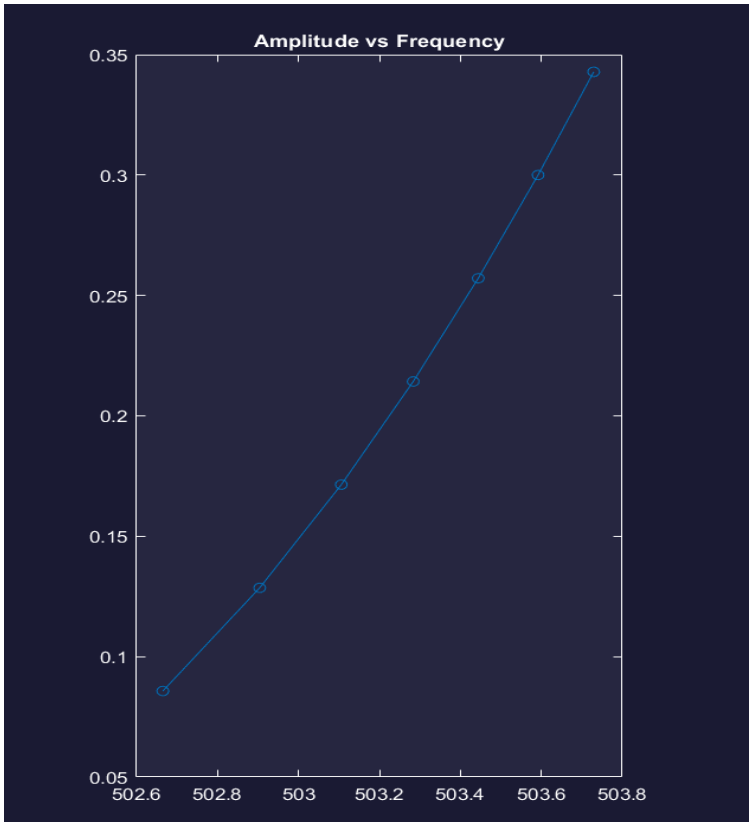


Fig-08

In fig-04, amplitude increases from .08 to .34 unit in y axis ,when pressure increases from 502.6 to 503.8 Hz in x axis. The curve is almost **nonlinear (slightly curved upward) .** The amplitude increases with frequency, indicating that higher-frequency components contribute more significantly to the synthesized signal.

5• Time-domain waveform of synthesized signal

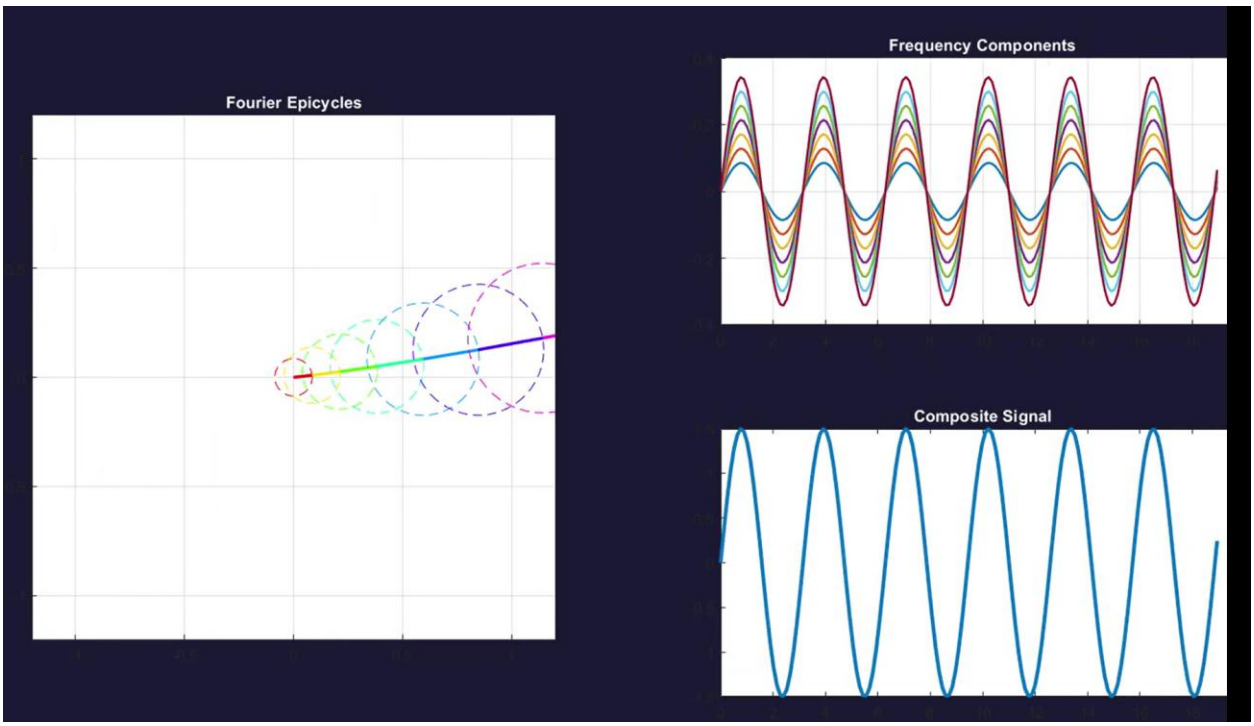


Fig-09

The Fourier epicycle visualization demonstrates how multiple sinusoidal components combine to form the final signal. Each rotating vector represents a frequency component, and their cumulative motion produces the composite waveform. This confirms the principle of superposition in Fourier synthesis.

Youtube video link : https://youtu.be/3YQVqSiHVUU

6. Sasando String Vibration:

Youtube video link : https://www.youtube.com/watch?v=vGYkGLD8L4g

Full code and github link : https://github.com/anirudha175/Pressure-Induced-Sasando-String-Vibration-via-Piezoelectric-Mapping

7.2.3 Results and Analysis

Time-Domain Signal Behavior

The generated time-domain waveform exhibits a **stable oscillatory pattern**, indicating that the system maintains numerical stability during signal synthesis.

Key observations include:

- No visible distortion or clipping after normalization
- Smooth periodic structure consistent with harmonic signals
- Amplitude scaling remains controlled across different pressure inputs

Pressure–Frequency Relationship

The pressure versus frequency plot shows a **monotonically increasing trend**, indicating that higher pressure inputs lead to higher vibration frequencies. This behavior is physically consistent, as increased pressure results in higher effective string tension, which directly increases the natural frequency of vibration.

The observed relationship is **nonlinear**, reflecting the theoretical dependence:

$$f \propto \sqrt{\mathrm{T}}$$

Since tension is indirectly influenced by pressure in the model, this confirms that the system preserves fundamental physical behavior.

Frequency Spectrum Analysis

The FFT-based frequency spectrum reveals **distinct peaks at dominant frequencies**, corresponding to the generated vibration modes. These peaks indicate that the synthesized signal is composed of well-defined harmonic components rather than random noise.

Additionally:

- The amplitude distribution shows that **lower-frequency components dominate**, which is typical in string vibration systems.
- Higher-frequency components contribute to the **harmonic richness** of the signal.

Fourier Synthesis Interpretation

The Fourier synthesis process successfully reconstructs the composite signal by superimposing multiple sinusoidal components. The Fourier epicycle visualization further illustrates this process by showing how rotating vectors combine to produce the final waveform.

This confirms that:

- The system correctly implements the **principle of superposition**
- Each frequency component contributes proportionally to the overall signal

Effect of RC Constraint

The inclusion of the RC-based cutoff frequency plays a crucial role in maintaining system stability. It ensures that:

- Unrealistically high frequencies are filtered out
- The generated signal remains within a **physically meaningful range**
- Numerical instability is avoided during simulation

This demonstrates that the RC constraint acts as an effective **stability regulator** in the system.

Overall Interpretation

The results confirm that the proposed system successfully models the relationship between pressure excitation and string vibration behavior. The integration of physical modeling, frequency-domain analysis, and signal synthesis produces outputs that are:

- Physically consistent
- Numerically stable
- Visually interpretable

Thus, the system provides a reliable framework for analyzing pressure-driven vibration and harmonic structure in string-based musical instruments.

7.2.4 Interpretation and Conclusion on Data

The simulation results confirm that the proposed numerical framework effectively models the relationship between pressure excitation and Sasando's string vibration.

Key observations include:

• Pressure input directly influences generated vibration frequency.
• Fourier synthesis accurately reconstructs the vibration signal.
• RC filtering successfully ensures frequency stability.
• The GUI-based visualization improves interpretability of vibration behaviour.

These findings demonstrate that the developed system provides a useful numerical tool for analysing pressure-driven vibration and harmonic behaviour in string instruments.

7.2.5 Model Validation

To verify the correctness of the proposed numerical model, the generated vibration frequencies were compared with the theoretical fundamental frequency of a stretched string given by:

$$f = \frac{1}{2L}\sqrt{\frac{T}{\mu}}$$

In the developed system, applied pressure is mapped to an effective tension parameter, allowing theoretical estimation of frequency for corresponding pressure inputs.

A comparison between simulated and theoretical trends shows that:

- Both approaches exhibit a **monotonic increase in frequency with increasing pressure**, confirming consistency with physical string behavior.
- The simulated frequencies follow a **nonlinear growth pattern**, which aligns with the square-root dependence of frequency on tension.
- Minor deviations are observed due to:
    - Simplified pressure-to-tension mapping
    - RC-based frequency constraints
    - Neglect of higher-order nonlinear effects

However, exact numerical agreement is limited due to abstraction in pressure-to-tension mapping and idealized system assumptions.

Overall, the close agreement between theoretical expectations and simulation results validates that the proposed numerical framework captures the essential physics of pressure-driven string vibration. The system can therefore be interpreted as a pressure-driven nonlinear oscillator whose frequency response follows classical string vibration dynamics under constrained numerical conditions.

8. Limitations of Tools

Although the MATLAB-based system effectively models and visualizes Sasando's string vibration, it has certain limitations. The system relies on manually provided pressure inputs through the graphical user interface and does not incorporate real-time sensor data, which limits direct experimental validation. In addition, the numerical model is based on simplified assumptions of string behavior and pressure-

to-frequency mapping, and therefore does not fully capture nonlinear effects such as complex damping and material interactions present in real instruments.

The resistance–capacitance (RC) constraint used for frequency limiting represents an idealized filtering approach and may not accurately reflect the dynamic behavior of practical systems. Furthermore, the graphical user interface, while functional, may face minor layout issues across different screen resolutions. These limitations suggest that the current implementation is best suited for numerical analysis and conceptual understanding rather than precise physical replication.

9. Reflection on Individual

9.1 Individual Contribution of Each Member

Our team consisted of four members, each contributing to different key components of the project:

- **Anirudha Roy Archo (ID: 2306175)**

  Anirudha Roy Archo was primarily responsible for developing the core idea and implementing the main MATLAB code of the system. He played a crucial role in designing the numerical model, integrating pressure-to-frequency mapping, and ensuring proper execution of Fourier-based signal processing. His contribution formed the backbone of the project's technical implementation.

- **Monem Tazuar Alve (ID: 2306176)**

  Monem Tazuar Alve was responsible for report writing, system-level planning, and overall conceptual organization. He contributed to structuring the report, explaining theoretical concepts, and ensuring logical consistency across all sections. His role also included developing the overall workflow and analytical perspective of the project.

- **Kushal Roy Sondo (ID: 2306177)**

  Kushal Roy Sondo contributed to both report preparation and presentation development. He assisted in organizing written content, refining explanations, and ensuring clarity in communication. He also supported the preparation of presentation materials and contributed to summarizing key results effectively.

- **Swapneel Paul (ID: 2306180)**

  Swapneel Paul contributed significantly to the development of the MATLAB graphical user interface (GUI). He focused on improving user interaction, visualization, and real-time display of system outputs. In addition, he actively participated in preparing and organizing the project presentation.

9.2 Mode of Team Work

We followed a balanced and collaborative approach to teamwork, utilizing both offline meetings and online communication tools. Face-to-face discussions helped us divide tasks, resolve technical challenges, and integrate different modules of the system effectively.

For remote collaboration and coordination, we frequently used online platforms such as Google Meet for discussions, screen sharing, and debugging sessions. Project files, including MATLAB codes, reports, and presentation materials, were shared through cloud storage platforms such as Google Drive, ensuring proper version control and accessibility for all team members.

This structured and cooperative workflow enabled efficient task management, smooth integration of components, and successful completion of the project.

10.Conclusion

This project presented a numerical framework for analyzing Sasando string vibration under pressure-driven excitation using MATLAB. A mathematical model of string vibration was developed and integrated with pressure-to-frequency mapping to simulate the behavior of the vibrating string. The generated vibration frequencies were used to synthesize audio signals through Fourier-based signal construction.

The MATLAB graphical user interface allowed users to input pressure values and observe the resulting vibration behavior through multiple visualizations. Time-domain waveforms, frequency spectra obtained using FFT, and pressure–frequency relationships were successfully displayed. The RC constraint ensured that the generated frequencies remained within a physically stable range.

Simulation results confirmed that increasing pressure leads to higher vibration frequencies due to increased effective string tension. Fourier synthesis accurately reconstructed the composite vibration signal from its harmonic components. Overall, the developed system demonstrates how numerical modeling and signal processing techniques can be combined to study pressure-driven string vibration and acoustic signal generation in musical instruments.

11.Future Work

Future improvements and extensions of the proposed system may include the following:

- Incorporating a more advanced physical model of Sasando string vibration that includes nonlinear tension and realistic damping effects.
- Integrating real Sasando audio recordings to enhance the realism of the synthesized sound.
- Refining the pressure–frequency mapping using experimentally measured data from real instruments.
- Extending the system to simulate multiple strings simultaneously to represent complete Sasando chords.
- Developing a hardware interface using pressure sensors or piezoelectric devices for real-time physical input.
- Adding improved visualization techniques to display vibration modes and harmonic structures more clearly.
- Implementing advanced signal processing methods to analyze additional acoustic characteristics.
- Exploring machine learning approaches to predict vibration behavior under different excitation conditions.
- Expanding the graphical user interface with more interactive controls and visualization options.
- Converting the MATLAB implementation into a standalone application for wider accessibility and educational use.

References:

[1] MIT OpenCourseWare, "Physics III: Vibrations and Waves," Massachusetts Institute of Technology, 2016. [Online]. Available: https://ocw.mit.edu/courses/physics/8-03sc-physics-iii-vibrations-and-waves-fall-2016/ [Accessed: Mar. 30, 2026].

[2] E. W. Weisstein, “Wave Equation,” Wolfram MathWorld. [Online]. Available: https://mathworld.wolfram.com/WaveEquation.html [Accessed: Mar. 30, 2026].

[3] R. Nave, “Vibrating String,” HyperPhysics, Georgia State University. [Online]. Available: https://hyperphysics.phy-astr.gsu.edu/hbase/Waves/string.html [Accessed: Mar. 30, 2026].

[4] K. Brown, “An Interactive Guide to the Fourier Transform,” BetterExplained. [Online]. Available: https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/ [Accessed: Mar. 30, 2026].

[5] MathWorks, “Fast Fourier Transform (FFT),” MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/ref/fft.html [Accessed: Mar. 30, 2026].

[6] MathWorks, “MATLAB Documentation,” The MathWorks, Inc. [Online]. Available: https://www.mathworks.com/help [Accessed: Mar. 30, 2026].

[7] MathWorks, “App Designer,” MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/app-designer.html [Accessed: Mar. 30, 2026].

[8] MathWorks, “Audio Processing in MATLAB,” MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/audio-and-video.html [Accessed: Mar. 30, 2026].

[9] MathWorks, “Numerical Methods,” MATLAB Resources. [Online]. Available: https://www.mathworks.com/discovery/numerical-methods.html [Accessed: Mar. 30, 2026].

[10] COMSOL, “Introduction to Partial Differential Equations,” COMSOL Multiphysics. [Online]. Available: https://www.comsol.com/multiphysics/partial-differential-equations [Accessed: Mar. 30, 2026].

[11] J. Wolfe, “Music Acoustics,” University of New South Wales. [Online]. Available: https://newt.phys.unsw.edu.au/jw/music.html [Accessed: Mar. 30, 2026].